

Code Guide for Multi-View Bayesian Modeling for Personalized Therapy Selection in High-Dimensional Data

Lorenzo Moni, Silvia Liverani, Alberto Cassese, Francesco Claudio Stingo

2026-06-08

The t-MVBPR model for treatment selection is mainly implemented in C++ in the file `<cppCodeMVBPR.cpp>`. The R function `tMVBPR()`, provided in `<RMultiTreatMVBPR.R>`, serves as a user-friendly interface to the C++ implementation: it performs input checks, prepares the required objects, and calls the underlying C++ function to estimate the model. In addition, `RMultiTreatMVBPR.R` contains auxiliary functions for post-processing and summarising the model output.

Function call

```
tMVBPR=function(dataset, predictiveDataset=NULL,
                Outcomemapping=NULL,
                NumberofViews=3,
                ListPrior=NULL,
                ListInitialValues=list(Numbinitialclusters=10),
                nameTreatment="Treatment",
                nameOutcome="Outcome",
                namesPredictive,
                namesPrognostic,
                MCMCfinalSampleSize,
                MCMCburnin,
                MCMCThinning,
                ListMCMCparam=list(PropVarTheta=0.1, InitVarBeta=2, RelViewNeal8Mparam=10),
                HowgammasInit=-2
)
```

<i>Argument</i>	<i>Description</i>
<code>dataset</code>	R dataframe used to fit the model. It must contain the treatment variable, the outcome variable, and the predictive and prognostic variable.
<code>predictiveDataset</code>	Optional data frame of new observations for posterior prediction, conditional on the predictive and prognostic variables. If <code>NULL</code> , predictions are computed for the dataset used to fit the model
<code>Outcomemapping</code>	Optional named vector used to map outcome labels to numerical values. If <code>NULL</code> , the outcome is assumed to be already coded appropriately.
<code>NumberofViews</code>	Number views. Default value 3.
<code>ListPrior</code>	Optional named list specifying prior distributions. If <code>NULL</code> , default prior settings are used and generated by <code>genDefaultPrior()</code> .

<i>Argument</i>	<i>Description</i>
<code>ListInitialValues</code>	Named list containing the initial clusters used to initialize the model. Default value: <code>list(Numbinitialclusters=10)</code> .
<code>nameTreatment</code>	Name of the column containing the treatment variable. Default value: “Treatment”.
<code>nameOutcome</code>	Name of the column containing the outcome variable. Default value: “Outcome”.
<code>namesPredictive</code>	Character vector containing the names of the predictive covariates.
<code>namesPrognostic</code>	Character vector containing the names of the prognostic covariates.
<code>MCMCfinalSampleSize</code>	Number of posterior samples retained after burn-in and thinning.
<code>MCMCburnin</code>	MCMC burn-in period.
<code>MCMCThinning</code>	Thinning interval for the MCMC chain.
<code>ListMCMCparam</code>	Named list containing the MCMC tuning parameters: the proposal variance for <code>theta</code> , the initial variance for the regression coefficients of the prognostic variables, and the <code>M</code> parameter of Neal’s Algorithm 8 for the relevant view. Default value: <code>list(PropVarTheta = 0.1, InitVarBeta = 2, RelViewNeal8Mparam = 10)</code> .
<code>HowgammasInit</code>	Type of initialization for the gamma variables. If <code>-2</code> , gamma variables are initialized using a frequentist preprocessing step. If <code>-1</code> , they are initialized randomly. If <code>k</code> , with <code>k < NumberOfViews - 1</code> , all gamma variables are initialized to <code>k</code> . Default value: <code>-2</code> .

Exemple

In this example, we show the usage of the main `tMVBPR()` function and some other function implemented in `RMultiTreatMVBPR.R`.

We consider dataset of 300 units (150 per treatment) with 100 predictive variables, named `d1`, `...`, `d100`, and 3 prognostic variables, named `w1`, `...`, `w3`. The `predictiveDataset` contains 200 units.

Usage `tMVBPR()`

Both `dataset` and `predictiveDataset`, if provided, must be R data frames.

Predictive variables must be factors. If `predictiveDataset` is provided, factor levels must be consistent between `dataset` and `predictiveDataset`, even when some levels are not observed in one of the two data frames.

Prognostic variables can be numeric or factors. For factors with more than two levels, the user may want to explicitly set the reference level, since the model includes `nlevels - 1` coefficients with respect to that reference category.

The outcome variable must be a factor. We recommend specifying `Outcomemapping`, since otherwise the function uses the order of the outcome factor levels to define the numerical coding. Note also that last outcome category is used as the reference category in the model.

The treatment variable must be numeric and coded with integer values from 1 to T, where T is the number of treatments. No gaps in the treatment labels are allowed.

```
rm(list = ls())

source("RMultiTreatMVBPR.R", echo = FALSE)

testlst=readRDS("Testdata")

testdata=testlst$data # nobstotal =nrow(testdata)=300
testdataPred=testlst$datapred #ntilde=nrow(testdataPred)=200

# In this example, we relevele the categorical prognostic variable w3.
# The same ref. must be applied to both datasets.
testdata$w3=relevel(droplevels(factor(testdata$w3)), ref = "1")
testdataPred$w3=relevel(droplevels(factor(testdataPred$w3)), ref = "1")

#1st MCMC chain
set.seed(2214)
C1=tMVBPR(dataset = testdata, predictiveDataset=testdataPred,
  Outcomemapping=c("CR"=2,"PD"=0,"SD"=1),
  NumberofViews=4,
  ListPrior=NULL,
  ListInitialValues=list(Numbinitialclusters=10),
  nameTreatment="asstrt",
  nameOutcome="outcome",
  namesPredictive=paste0("d",1:100),
  namesPrognostic=c("w1","w2","w3"),
  MCMCfinalSampleSize=10000,
  MCMCburnin=100000,
  MCMCThinning=50,
  ListMCMCparam=list(PropVarTheta=0.05, InitVarBeta=2, RelViewNeal8Mparam=15),
  HowgammasInit=-2
)

## 10% complete20% complete30% complete40% complete50% complete60% complete70% complete80% complete90%

#2nd MCMC chain
set.seed(1245)
C2=tMVBPR(dataset = testdata, predictiveDataset=testdataPred,
  Outcomemapping=c("CR"=2,"PD"=0,"SD"=1),
  NumberofViews=4,
  ListPrior=NULL,
  ListInitialValues=list(Numbinitialclusters=10),
  nameTreatment="asstrt",
  nameOutcome="outcome",
  namesPredictive=paste0("d",1:100),
  namesPrognostic=c("w1","w2","w3"),
  MCMCfinalSampleSize=10000,
  MCMCburnin=100000,
  MCMCThinning=50,
  ListMCMCparam=list(PropVarTheta=0.05, InitVarBeta=2, RelViewNeal8Mparam=15),
  HowgammasInit=-2
)
```

```

)

## 10% complete20% complete30% complete40% complete50% complete60% complete70% complete80% complete90% complete
# Important: store the results of each chain in a named list using the names Chain1, ...,
# ↪ ChainK.
# This structure is required by the MCMC diagnostic tools implemented in the
# ↪ RMultiTreatMVBPR.R script
Results=list(Chain1=C1, Chain2=C2)

```

MCMC Diagnostic

```

# The following functions can be used to assess MCMC convergence.

# Assess convergence of the DP hyperparameters and prognostic effects.
# The function produces and saves two PDF files, Alphas.pdf and Betas.pdf,
# in the folder specified by pathtosaveplots.
dBA=DiagnosticBetaAlpha(Post = Results, pathtosaveplots = "~/Desktop/")

```

```

## Plotting density plots
## Plotting traceplots
## Plotting comparison of partial and full chain
## Plotting autocorrelation plots
## Plotting Potential Scale Reduction Factors
## Plotting Number of effective independent draws
## Plotting Geweke Diagnostic
## Plotting caterpillar plot
## Time taken to generate the report: 2.4 seconds.
## Plotting density plots
## Plotting traceplots
## Plotting comparison of partial and full chain
## Plotting autocorrelation plots
## Plotting Potential Scale Reduction Factors
## Plotting Number of effective independent draws
## Plotting Geweke Diagnostic
## Time taken to generate the report: 2.3 seconds.

```

```
dBA
```

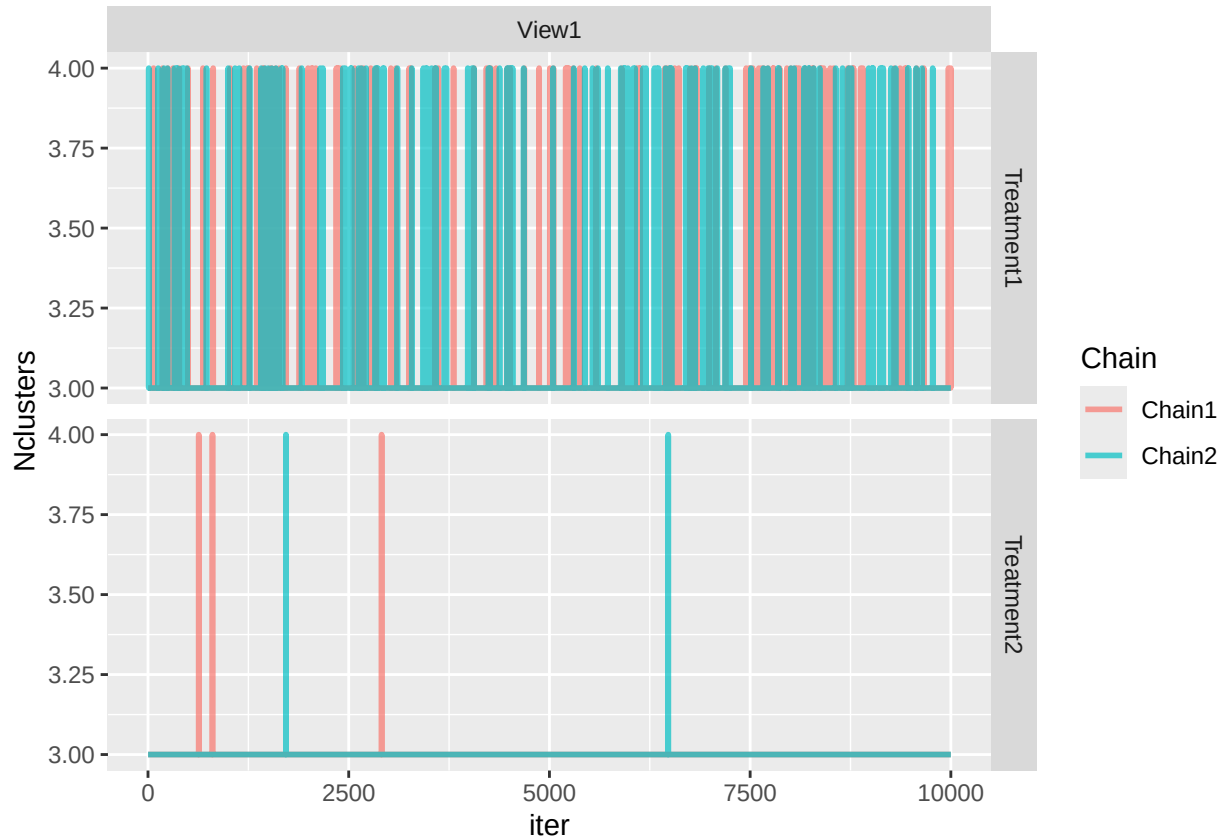
```

## # A tibble: 12 x 5
##   Parameter      z_chain1 z_chain2  Rhat Effective
##   <fct>          <dbl>    <dbl> <dbl>    <dbl>
## 1 beta_w1_r1    -0.112   -1.24  1.00    0.411
## 2 beta_w1_r2    -1.57    -2.31  1.00    0.447
## 3 beta_w2_r1     1.43     0.880  1.00    0.798
## 4 beta_w2_r2     1.15     2.35  1.000   0.651
## 5 beta_w30_r1    1.95     0.494  1.00    0.465

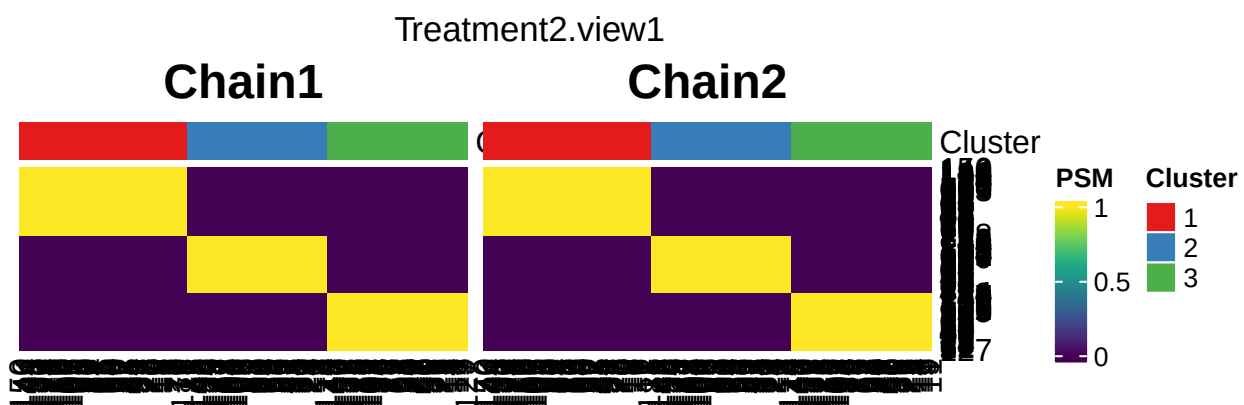
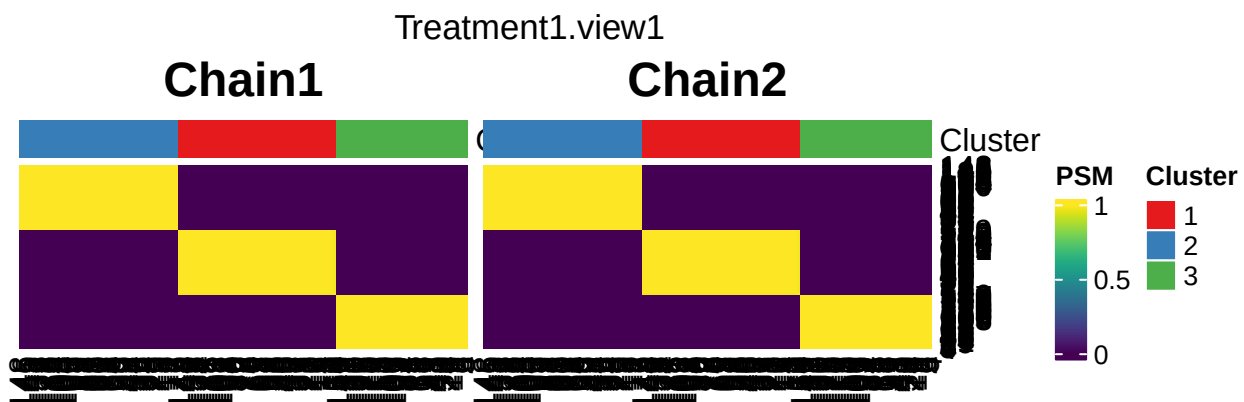
```

```
## 6 beta_w30_r2      2.03      0.329 1.00      0.572
## 7 Treatment1 alpha1 0.236      0.703 1.000      0.974
## 8 Treatment1 alpha2 0.559      0.828 1.05      0.201
## 9 Treatment1 alpha3 1.12      -0.290 1.06      0.169
## 10 Treatment2 alpha1 2.71      0.641 1.00      1.000
## 11 Treatment2 alpha2 -0.436     1.30  1.06      0.170
## 12 Treatment2 alpha3 1.09      -0.513 1.06      0.174
```

```
# Assess convergence of the cluster-allocation variables.
# The function plots the co-occurrence matrix for each chain
DiagnosticClusterallocations(PostMultipleChains = Results)
```



```
## # A tibble: 2 x 4
## # Groups:   Treatment, View [2]
##   Treatment View Chain1 Chain2
##   <chr>      <chr> <dbl> <dbl>
## 1 Treatment1 View1  3.01  3.01
## 2 Treatment2 View1  3.00  3.00
## [1] 3
## [1] 3
## [1] 3
## [1] 3
```



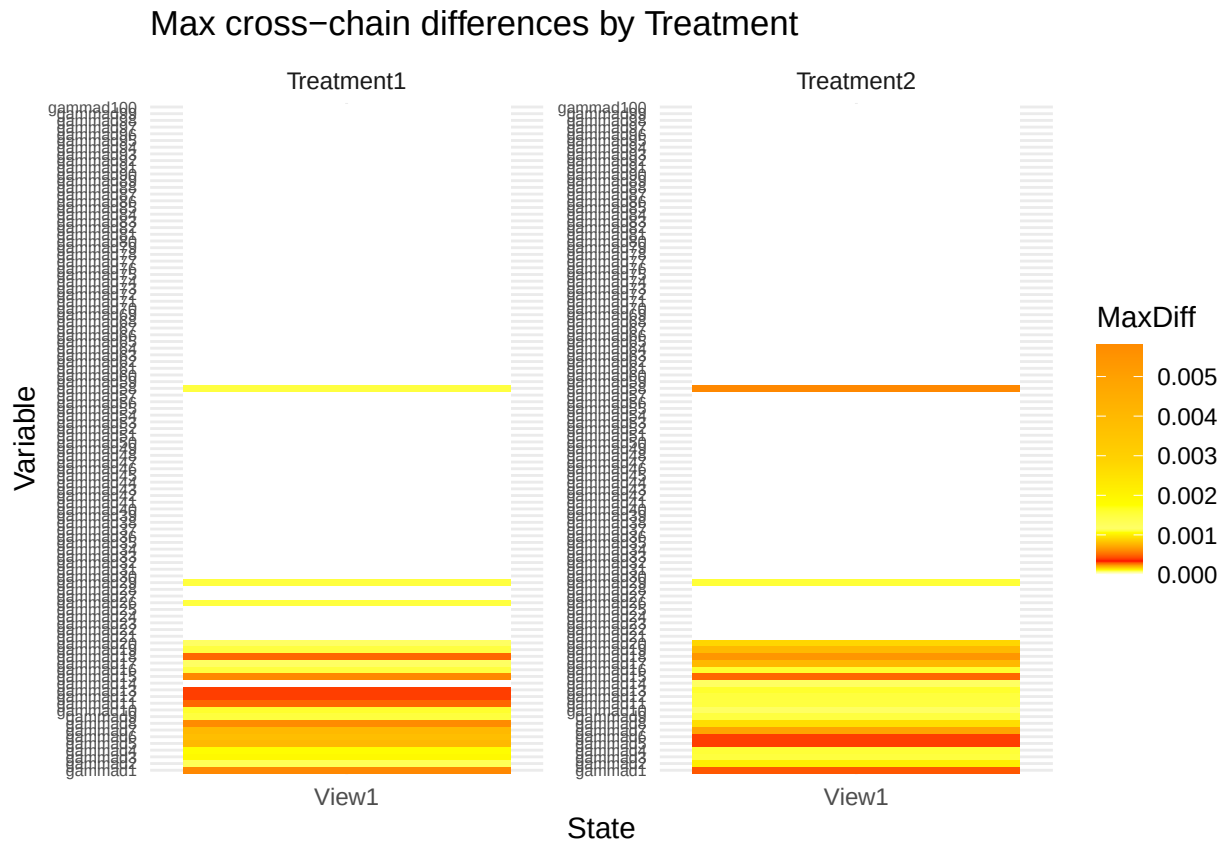
Assess convergence of the view-allocation variables for the relevant view.
The function plots the maximum difference in marginal inclusion probabilities
across chains for each predictive variable

```
dG=DiagnosticViewAllocations(PostMultipleChains = Results )
```

```
## Treatment Treatment1 agreement between chains
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.50   1.00   1.00   0.99   1.00   1.00
## Treatment Treatment2 agreement between chains
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.500   1.000   1.000   0.995   1.000   1.000
##
## summary statistics of max difference Marginal probabilities:
## $Treatment1
##   View1
##   Min.   :0.000000
##   1st Qu.:0.000000
##   Median :0.000000
##   Mean   :0.000238
##   3rd Qu.:0.000000
##   Max.   :0.005800
##
## $Treatment2
##   View1
##   Min.   :0.000000
##   1st Qu.:0.000000
##   Median :0.000000
```

```
## Mean      :0.000261
## 3rd Qu.   :0.000000
## Max.      :0.005400
```

dG



Combining chains, treatment selection, and inference

```
# Combine multiple MCMC chains
ResultsComb=combineChains(Results)

# Check the final MCMC sample size.
# In this example, the final sample size is 20000 (each chain is 10000)
ResultsComb$Info$MCMCsamplesize

## [1] 20000

#Now we can use the final MCMC sample to perform treatment selection and inference

##TREATMENT SELECTION
# Estimate the optimal treatment using the final MCMC sample
# and the user-defined utility weights.
Predtrt=EstimOptTreatment(PostPredictiveAlltrt = ResultsComb, weights = c(0,40,100))

# Estimated optimal treatment for each predictive patient
```

```

head(Predtrt$EstimatedAssTrt)

## itilde=1 itilde=2 itilde=3 itilde=4 itilde=5 itilde=6
##          2          2          2          2          2          2

tail(Predtrt$EstimatedAssTrt)

## itilde=195 itilde=196 itilde=197 itilde=198 itilde=199 itilde=200
##           1           1           2           2           1           1

# Predicted outcome for each predictive patient, using the numerical outcome coding
head(Predtrt$PredOutcome)

## [1] 2 2 2 2 2 0

tail(Predtrt$PredOutcome)

## [1] 0 2 0 2 2 1

# Utility value for each predictive patient and treatment
head(Predtrt$Utility)

##           [,1]    [,2]
## itilde=1 87.400 99.861
## itilde=2 67.649 98.670
## itilde=3 77.416 99.560
## itilde=4 68.942 99.562
## itilde=5 42.824 90.798
## itilde=6  5.507 26.489

tail(Predtrt$Utility)

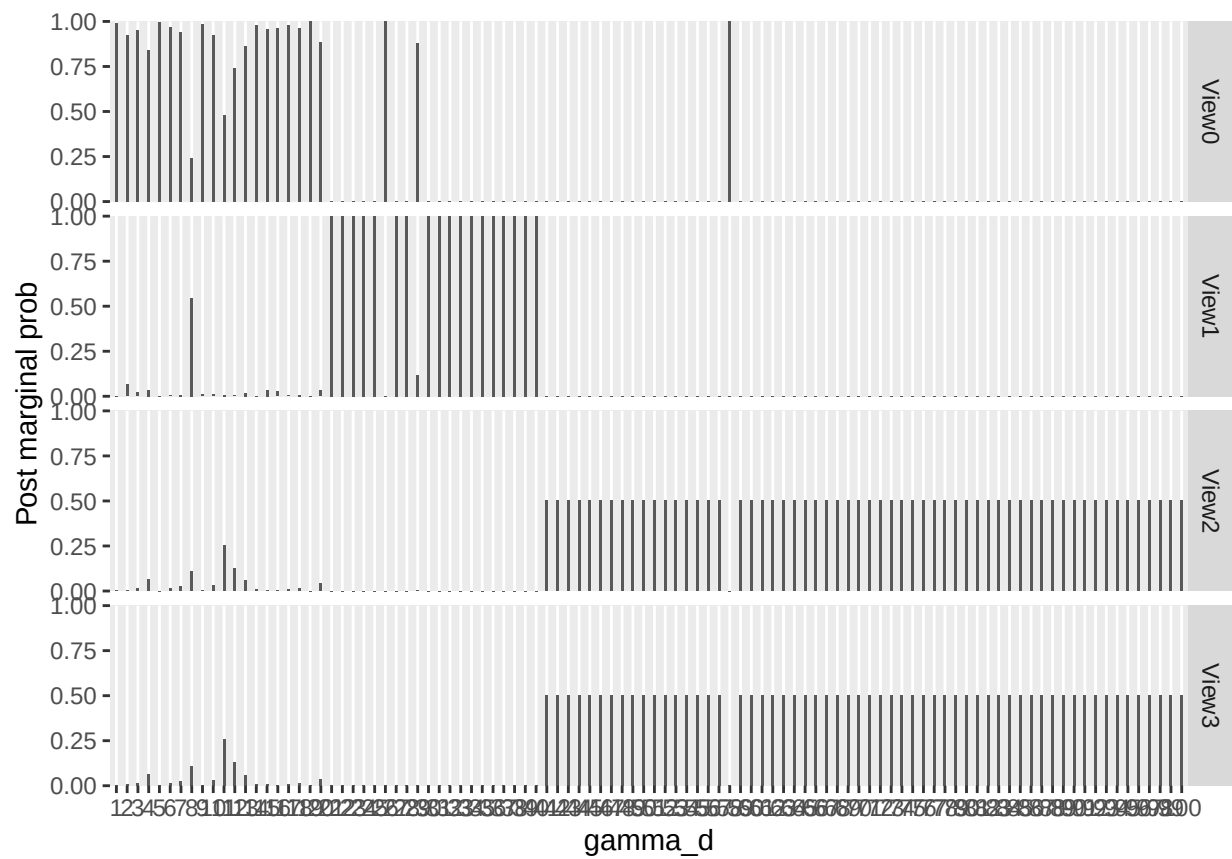
##           [,1]    [,2]
## itilde=195 15.343 11.808
## itilde=196 91.192 29.649
## itilde=197  0.478 27.321
## itilde=198 87.923 99.872
## itilde=199 99.347 48.087
## itilde=200 21.624 12.700

##CLUSTERING
# Compute the optimal posterior partition in each view for treatment 1
Zopttrt1=OptPostPart(ZarrPost = ResultsComb$Treatment1$ZZallviews)
head(Zopttrt1)

##      Zopt1 Zopt2 Zopt3
## [1,]     1     1     1
## [2,]     2     2     2
## [3,]     3     3     3
## [4,]     3     4     4
## [5,]     2     2     2
## [6,]     3     4     4

##VIEW ALLOCATION
# Plot posterior inclusion probabilities for the view-allocation variables.
# Example for treatment 1 (and type=1 plot)
PlotMPPViewAlloc(PostDraws = ResultsComb$Treatment1$Gammas,TypeofPlot = 1)

```

```
# Example for treatment 2 (and type=2 plot)
PlotMPPViewAlloc(PostDraws = ResultsComb$Treatment2$Gammas, TypeofPlot = 2)
```



““